INF384 · Temas Avanzados en Ingeniería de Software 1

Laboratorio 2

Pipeline con validación de calidad

Duración: 2 horas · **Modalidad:** individual · **Puntaje:** 20 puntos

---

Punto de partida

Su fork tiene un pipeline que funciona: instala dependencias, ejecuta pruebas, envía el resultado a SonarCloud y publica un paquete. Todas las ejecuciones terminan en verde.

El pipeline tiene **cuatro defectos**. Ninguno hace que falle.

Al terminar el laboratorio, el pipeline debe corregir los cuatro y publicar un artefacto versionado solo cuando el análisis de calidad lo permita.

Trabajen en su propio fork. Pueden consultarse entre ustedes; la entrega es personal.

Parte 1 — Diagnóstico (20 min)

Creen el archivo `docs/diagnostico.md` en su repositorio y respondan:

1.1 Los cuatro defectos. Para cada uno: qué está mal, en qué archivo y en qué líneas se manifiesta, y qué consecuencia tiene. Un defecto no es "falta una línea": es qué garantía se pierde por no tenerla.

1.2 El defecto que explica la duración. De los cuatro, cuál explica el tiempo que registraron en `docs/linea-base.md`. Sustenten con el número que midieron.

1.3 El vínculo con su caso. Cuál de los cuatro defectos ataca la restricción del caso transversal de su grupo. Citen un dato del value stream map que levantaron en la Sesión 1.

1.4 La métrica DORA. Qué métrica DORA esperan mover con la intervención y por qué. Solo dos son alcanzables sin despliegue: identifiquen cuáles y elijan una.

1.5 El proxy. Qué número concreto van a medir para sustentar que la métrica se movió. Declárelo **antes** de intervenir.

> Hagan commit de este archivo antes de tocar el workflow. La marca de tiempo del commit es parte de la evidencia.

Parte 2 — Intervención (55 min)

Corrijan los cuatro defectos en `.github/workflows/pipeline.yml`.

El pipeline resultante debe cumplir:

#	Condición
1	Las dependencias se instalan desde el archivo de bloqueo, no resolviendo versiones
2	Las dependencias se cachean entre ejecuciones
3	El pipeline se detiene si el análisis de calidad no cumple el quality gate
4	El artifact publicado debe llamarse `despachos-<versión>`, solo desde `main`, y solo si la validación pasó

Sobre el punto 4. La versión no se inventa: se deriva del historial de commits desde el tag `v1.2.0`. Revisen qué tipo de cambios hay desde ese tag y determinen si corresponde mayor, menor o parche. Actualicen `VERSION` y `pyproject.toml` con el valor que corresponda, y justifíquenlo en el entregable.

Sobre el quality gate. Configuren `sonar-project.properties` con su organization key y su project key antes de la primera ejecución.

Verificación. Ejecuten el pipeline y confirmen que pasa en verde y que el artefacto publicado lleva la versión en el nombre.

Parte 3 — Inyección de falla (20 min)

Hasta acá vieron el pipeline pasar. Ahora demuestren que **detiene**.

1. Creen una rama nueva desde `main`.
2. Agreguen a un archivo existente en `src/despachos/` una función nueva de al menos 15 líneas, con lógica real —condicionales, no un `return` fijo— y **sin ninguna prueba que la cubra**.
3. Abran un pull request hacia `main` de **su propio fork**. Verifiquen el *base repository* antes de crearlo.
4. El pipeline debe fallar en el paso de análisis de calidad, por cobertura insuficiente sobre código nuevo.
5. **No fusionen el pull request.** Guarden la URL de la ejecución fallida.

   Si el pipeline pasa en verde, el quality gate no está deteniendo nada. Revisen el punto 3 de la Parte 2.

Parte 4 — Cierre (15 min)

Agreguen a `docs/diagnostico.md`:

4.1 Medición posterior. El valor del proxy después de la intervención, junto al de la línea base. Qué cambió y en qué proporción.

4.2 Justificación de la versión. Qué versión declararon y qué commits del historial la sustentan.

4.3 Lo que no se resolvió. El pipeline sigue teniendo limitaciones. Nombren una y expliquen qué haría falta para resolverla.

4.4 Declaración de uso de IA generativa, conforme al sílabo.

Entrega

En PAIDEIA suban un archivo WORD con el siguiente nombre
"**INF384-LAB2-[Código-PUCP]", en el contenido coloquen los siguientes puntos:**

1. Screenshot y URL de `docs/diagnostico.md` completo
2. Screenshot y URL de la **ejecución fallida** del pull request de la Parte 3
3. Screenshot y URL de la **ejecución exitosa** en `main` que publicó el artefacto versionado